



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	Achmad Faiz Yudha Ramadhan
NPM	2428240113
Kelas	SI5C
Topik Tugas 1	Topik 9 — Bioskop: Film
Entitas utama	Film
Tautan repositori	https://github.com/FaizZzhue/si5c_kuis1_AchmadFaizYudhaRamadhan
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5C_[NIM]_AchmadFaizYudhaRamadhan.pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan

Bab	Isi	Bukti utama
Bab 3	Menjalankan server	1 screenshot terminal
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Topik 9 — Bioskop: Film
Entitas utama (tunggal)	Film
Alamat dasar	/movies
Field data	judul (string); genre (string); durasiMenit (number); ratingUsia (SU 13+ 17+ 21+); sutradara (string)
Field wajib saat POST	judul, genre, durasiMenit, ratingUsia
Query string untuk filter	genre
Data awal	3 data awal: Jejak Senja, Malam Terakhir, Petualangan Langit
Modul lain (jika ada)	Tidak ada

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/movies	Tidak	200	Ambil semua data film (dengan filter genre bila ada)
2	GET	/movies/:id	Tidak	200	Ambil satu film berdasarkan ID
3	POST	/movies	Ya	201	Tambah data film
4	PUT	/movies/:id	Ya	200	Ubah data film
5	DELETE	/movies/:id	Ya	200	Hapus data film; perilaku dipertahankan seperti Tugas 1

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.

[Screenshot instalasi belum diberikan — ambil 1 screenshot terminal yang memuat `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya.]

Gambar 1. Versi Node.js dan instalasi paket

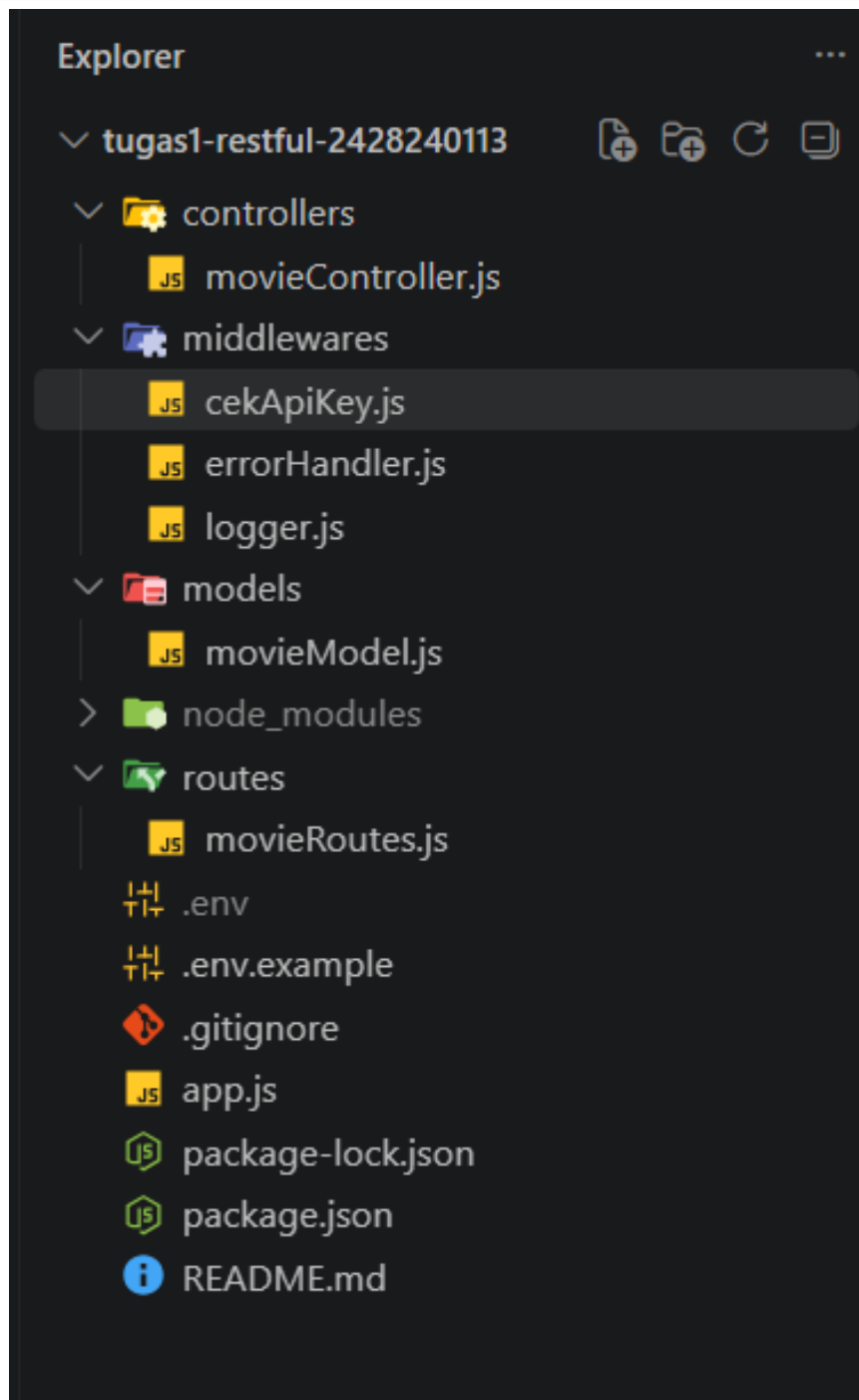
Penjelasan: Express digunakan untuk membangun server dan routing HTTP, CORS mengizinkan request lintas origin, sedangkan dotenv memuat pengaturan seperti PORT dan API_KEY dari berkas `.env`. Node.js diperlukan sebagai runtime JavaScript di sisi server dan kompatibel dengan Express 5 yang digunakan pada project ini.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan: Folder controllers menangani request, validasi, dan response; folder models menyimpan data film dan logika pengolahannya; folder routes memetakan endpoint ke controller; folder middlewares berisi logger, pemeriksaan API key, serta penanganan error.

2.2 Ringkasan tugas setiap berkas

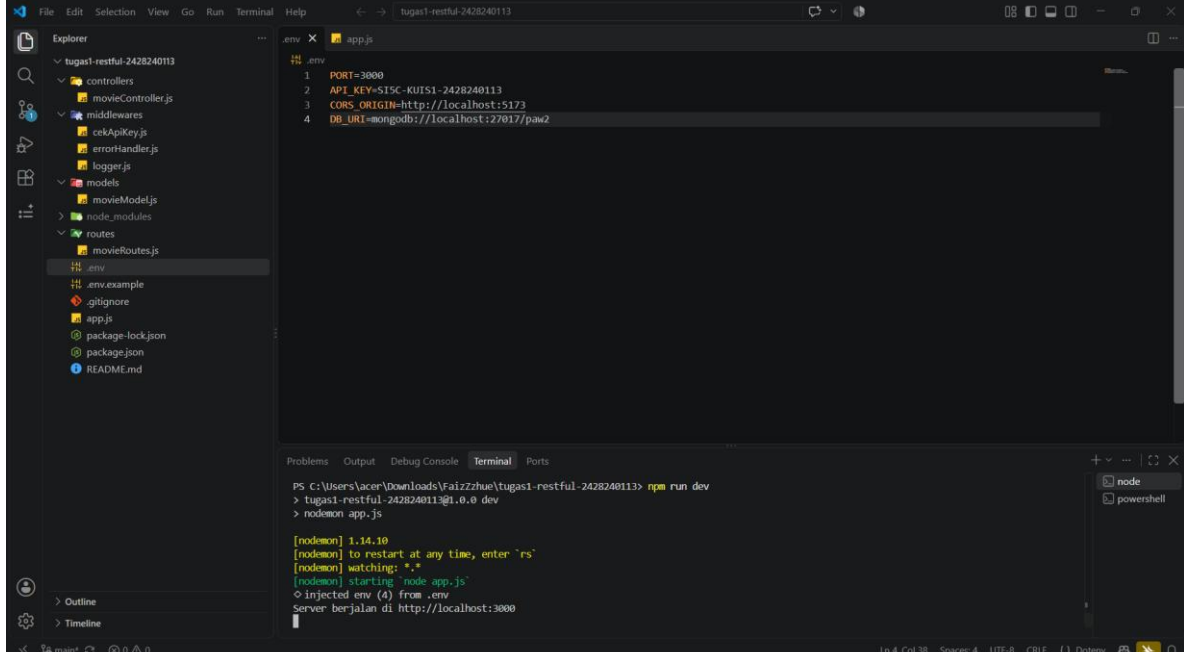
Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/[sumber-daya]Model.js	Menyimpan array movies dan nextId, serta fungsi getAllMovies, getMovieById, getMoviesByGenre, createMovie, updateMovie, dan deleteMovie tanpa menggunakan req/res.
controllers/[sumber-daya]Controller.js	Menangani request dan response untuk endpoint film. Fungsi validateMovie memeriksa field wajib, sedangkan getMovies, getMovieById, createMovie, updateMovie, dan deleteMovie mengatur alur HTTP.
routes/[sumber-daya]Routes.js	Membuat express.Router() dan memetakan GET, POST, PUT, dan DELETE ke fungsi controller. POST, PUT, dan DELETE dipasang middleware cekApiKey.
middlewares/logger.js	Mencatat waktu, method, dan URL setiap request melalui middleware logger sebelum request diteruskan dengan next().
middlewares/cekApiKey.js	Membaca x-api-key dari header dan membandingkannya dengan process.env.API_KEY. Request tanpa key menghasilkan 401, sedangkan key salah menghasilkan 403.
middlewares/errorHandler.js	Menangani JSON yang rusak serta error terpusat, termasuk mengubah route yang tidak ditemukan menjadi response JSON 404.
app.js	Memuat dotenv, mengaktifkan cors dan express.json(), memasang logger dan /movies routes, lalu mengaktifkan notFoundHandler, errorHandler, dan server HTTP.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



The screenshot shows a Visual Studio Code editor with a project named 'tugas1-restful-2428240113'. The Explorer sidebar on the left shows the project structure, including files like `controllers`, `middlewares`, `models`, `routes`, `env`, `env.example`, `gitignore`, `app.js`, `package-lock.json`, `package.json`, and `README.md`. The main editor area shows the `env` file with the following content:

```
1 PORT=3000
2 API_KEY=SI5C-KUISI-2428240113
3 CORS_ORIGIN=http://localhost:5173
4 DB_URI=mongodb://localhost:27017/paw2
```

The Terminal at the bottom shows the command `npm run dev` being executed, which starts the server using `nodemon app.js`. The output of the terminal is:

```
PS C:\Users\acer\Downloads\FaizZhu\Tugas1-restful-2428240113> npm run dev
> tugas1-restful-2428240113@1.0.0 dev
> nodemon app.js

[nodemon] 1.14.10
[nodemon] to restart at any time, enter "rs"
[nodemon] watching: *.*
[nodemon] starting: node app.js
O injected env (4) from .env
Server berjalan di http://localhost:3000
```

Penjelasan: Nomor port dibaca dari `process.env.PORT` yang dimuat oleh `dotenv` dari `.env`; pada project ini nilainya 3000. Jika port tersebut sedang dipakai aplikasi lain, Node.js tidak dapat melakukan bind ke port yang sama dan server akan gagal berjalan sampai port dikosongkan atau nilai `PORT` diganti.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

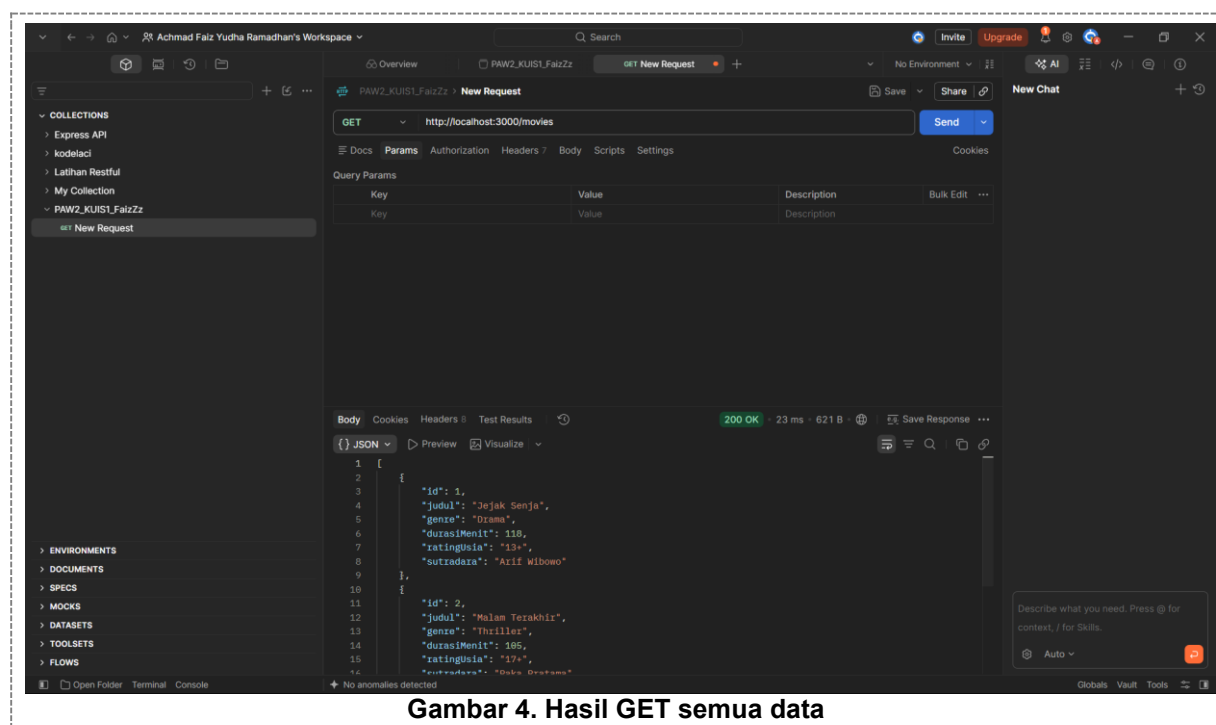
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[sumber-daya])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (request)	GET /movies
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; respons berupa larik JSON seluruh film.
Status hasil uji	200 OK — data film berhasil ditampilkan.
Lulus?	[X] Ya [] Tidak



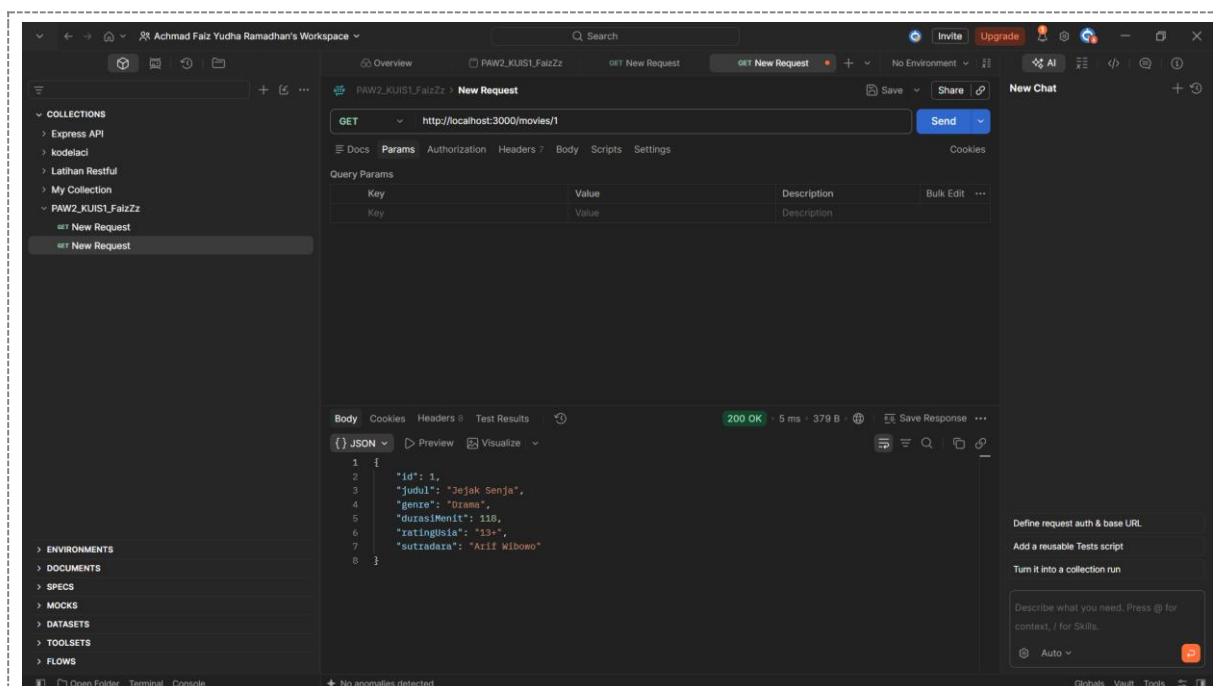
Gambar 4. Hasil GET semua data

Penjelasan: Status 200 menunjukkan request berhasil dan data film dikembalikan. Alurnya adalah route GET /movies memanggil movieController.getMovies, lalu controller memanggil movieModel.getAllMovies untuk memperoleh array film.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /movies/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id 1.
Status hasil uji	200 OK — film dengan id 1 berhasil ditampilkan.
Lulus?	[X] Ya [] Tidak



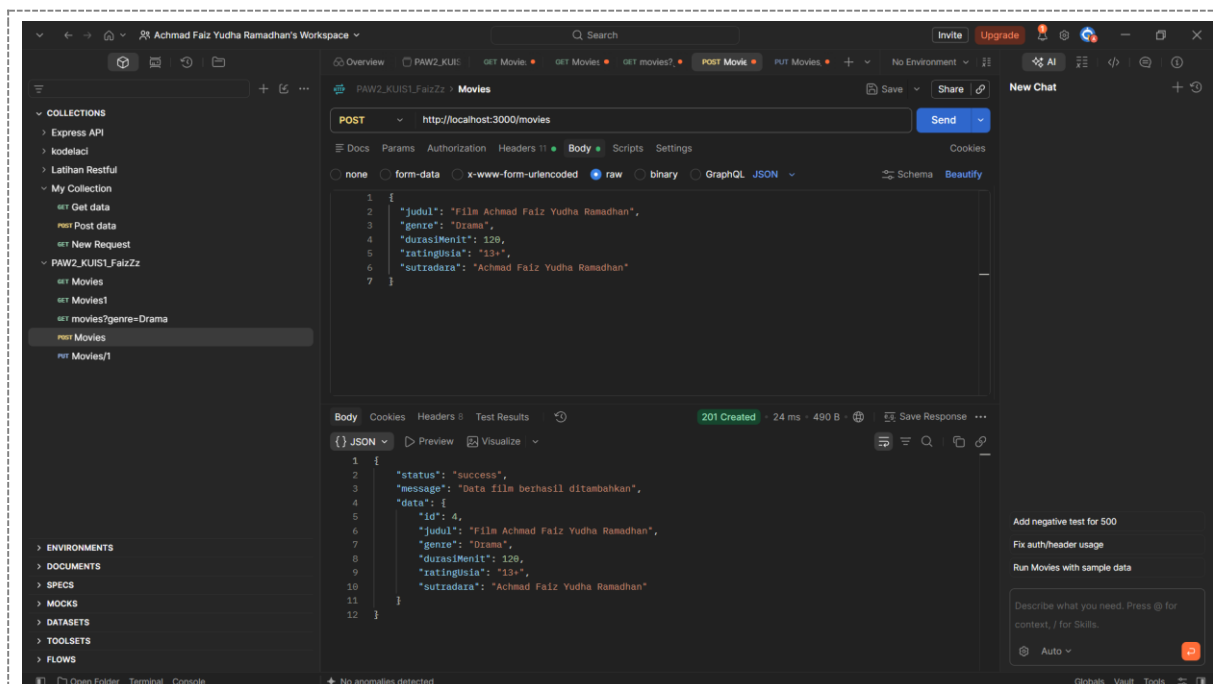
Gambar 5. Hasil GET berdasarkan ID

Penjelasan: ID diperoleh dari req.params.id pada URL /movies/1. Nilai dari URL berbentuk string sehingga dikonversi dengan parseInt sebelum dibandingkan dengan id numerik pada model.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /movies
Header	x-api-key: SI5C-KUIS1-2428240113 Content-Type: application/json
Body	{ "judul": "Film Achmad Faiz Yudha Ramadhan", "genre": "Drama", "durasiMenit": 120, "ratingUsia": "13+", "sutradara": "Achmad Faiz Yudha Ramadhan" }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created — data baru dibuat dengan id 4.
Lulus?	[X] Ya [] Tidak



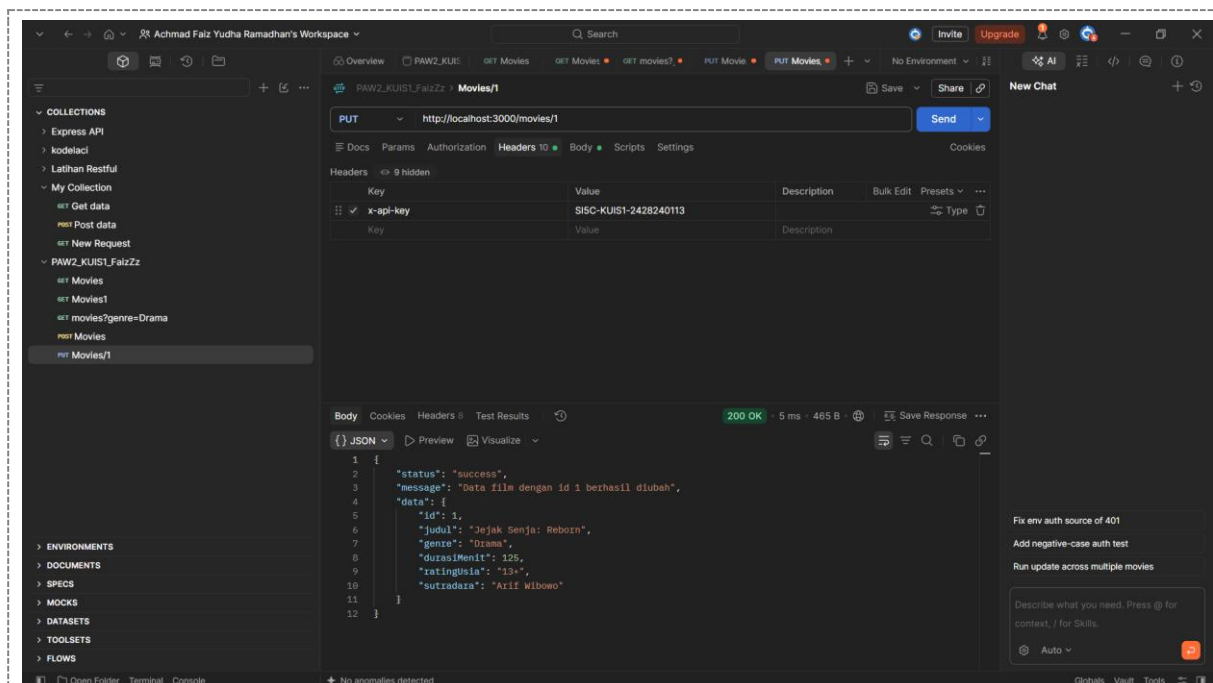
Gambar 6. Hasil POST data baru

Penjelasan: Header x-api-key diperiksa oleh middleware cekApiKey sebelum controller dijalankan. express.json() mengubah body JSON menjadi req.body. Status 201 digunakan karena request berhasil membuat resource film baru, bukan hanya membaca atau mengubah data.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /movies/1
Header	x-api-key: SI5C-KUIS1-2428240113 Content-Type: application/json
Body	{ "judul": "Jejak Senja: Reborn", "genre": "Drama", "durasiMenit": 125, "ratingUsia": "13+", "sutradara": "Arif Wibowo" }
Hasil yang diharapkan	200 OK; objek dengan data yang sudah berubah.
Status hasil uji	200 OK — data film id 1 berhasil diubah.
Lulus?	☒ Ya ☐ Tidak



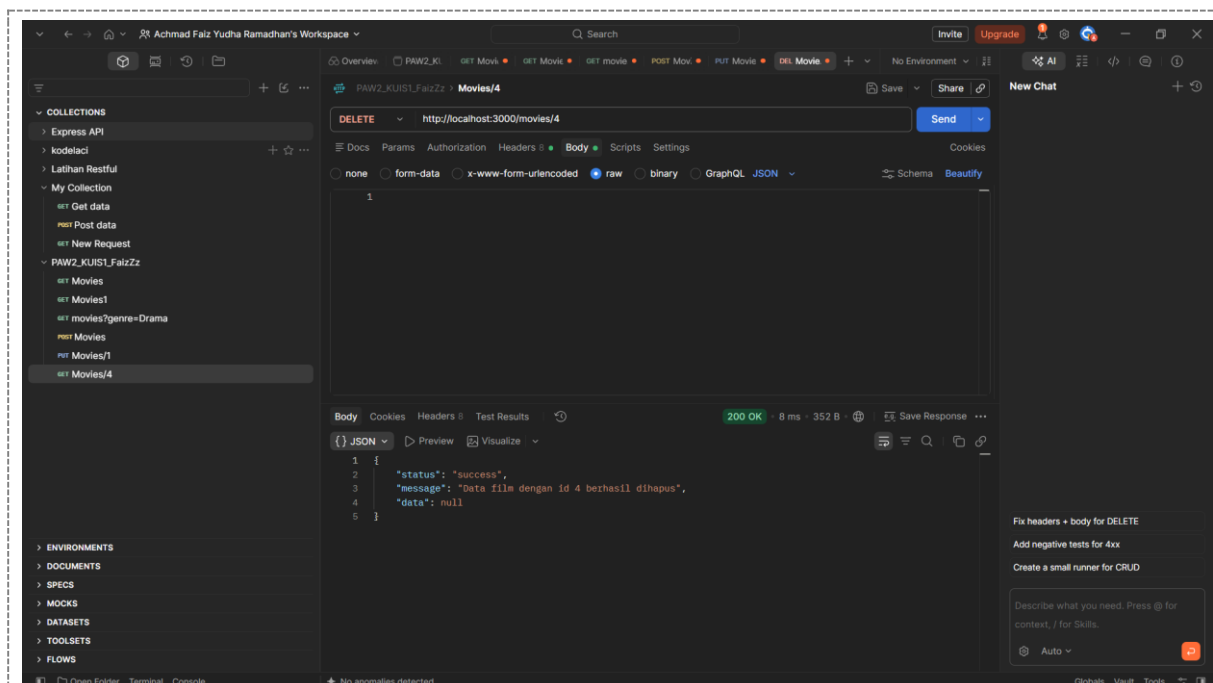
Gambar 7. Hasil PUT mengubah data

Penjelasan: Pada pengujian ini judul diubah menjadi "Jejak Senja: Reborn" dan durasi menjadi 125 menit, sementara genre, ratingUsia, dan sutradara tetap digunakan sebagai nilai lengkap. Controller melakukan validasi, kemudian `movieModel.updateMovie` mengganti data berdasarkan id.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 200, dan respons tanpa isi.

Permintaan (request)	DELETE /movies/4
Header	x-api-key: SI5C-KUIS1-2428240113
Body	(tidak ada)
Hasil yang diharapkan	200 OK dan data null, mengikuti perilaku API Tugas 1 yang dipertahankan setelah refactor.
Status hasil uji	200 OK — data film id 4 berhasil dihapus. Template contoh mencantumkan 204, tetapi implementasi ini mempertahankan response 200 dari Tugas 1.
Lulus?	☒ Ya ☐ Tidak



Gambar 8. Hasil DELETE data

Penjelasan: Status 200 adalah hasil aktual API karena refactor mempertahankan perilaku DELETE dari Tugas 1. Controller memanggil `movieModel.deleteMovie` untuk menghapus id 4 dan mengembalikan data null.

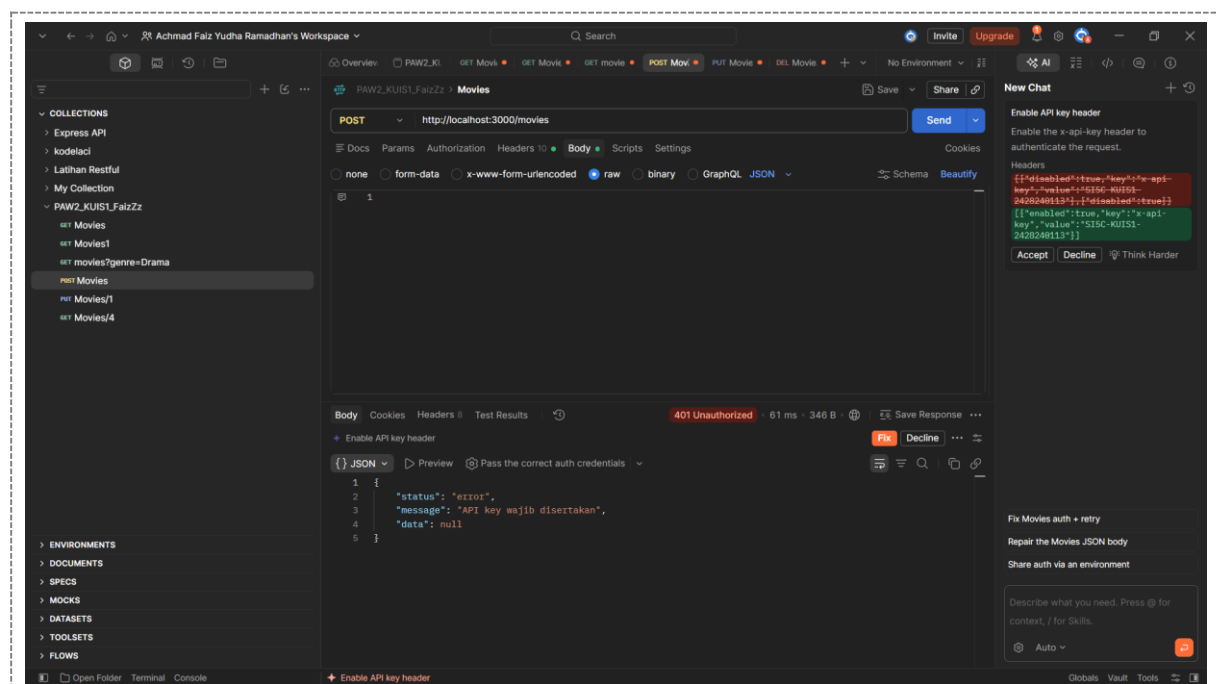
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /movies (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ data film valid }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	Bukti POST tanpa API key belum tersedia pada screenshot yang diberikan. Request gagal 401 pada pengujian PUT tanpa API key, tetapi template mensyaratkan POST untuk N01.
Lulus?	☒ Ya ☐ Tidak

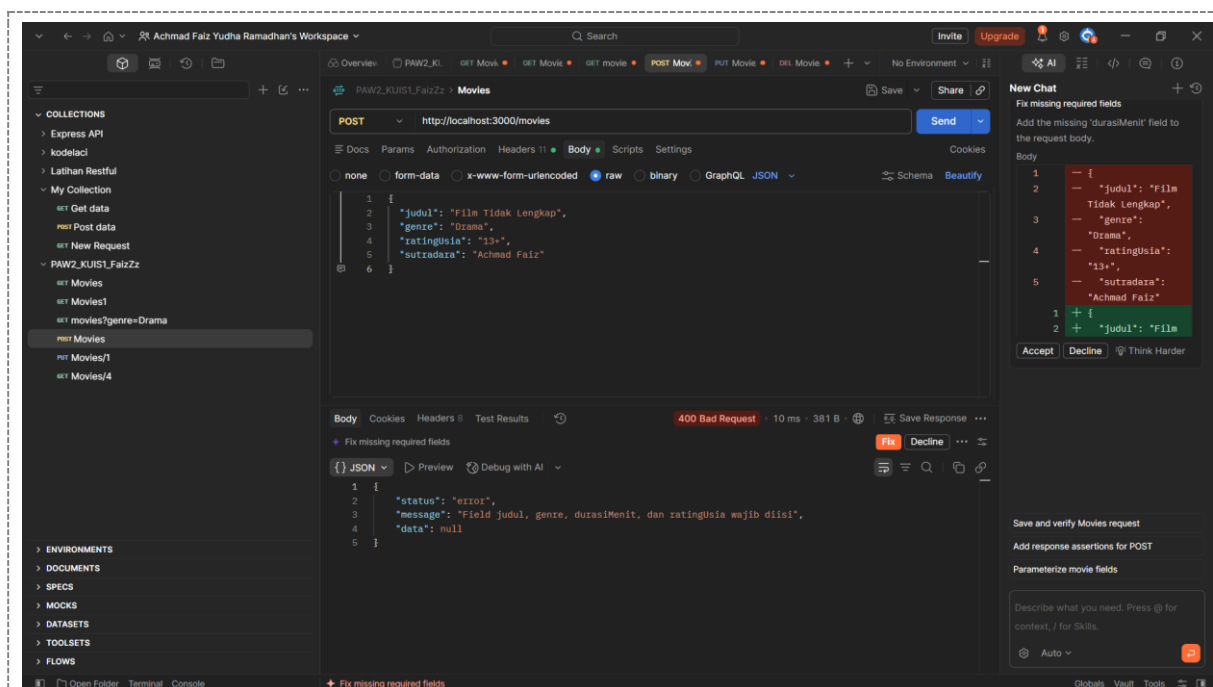


Penjelasan: Middleware cekApiKey menolak request sebelum controller dijalankan karena header x-api-key tidak ada. Itulah sebabnya response berhenti pada 401 dan controller createMovie tidak sempat melakukan validasi atau menyimpan data.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /movies
Header	x-api-key: SI5C-KUIS1-2428240113 Content-Type: application/json
Body	{ "judul": "Film Tidak Lengkap", "genre": "Drama", "ratingUsia": "13+", "sutradara": "Achmad Faiz" }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field wajib.
Status hasil uji	400 Bad Request — durasiMenit wajib diisi.
Lulus?	[X] Ya [] Tidak



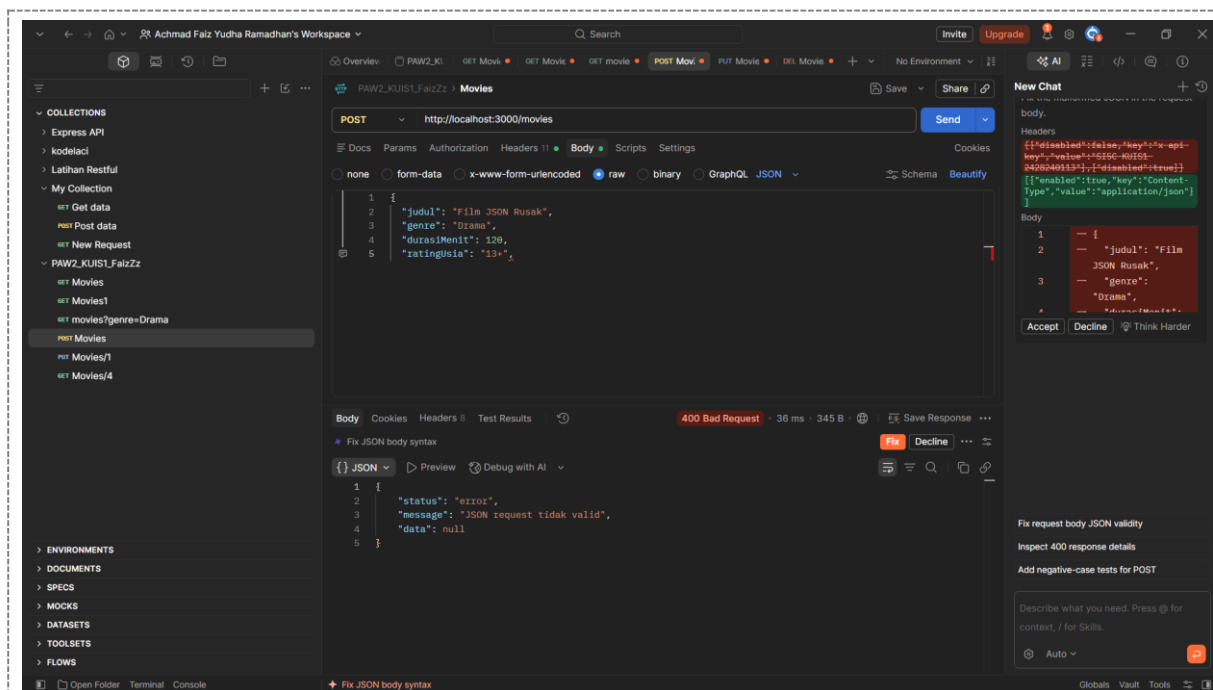
Gambar 10. POST dengan body tidak lengkap

Penjelasan: Validasi ditulis pada Controller melalui fungsi `validateMovie`. Karena request sudah melewati API key tetapi data wajib tidak lengkap, server mengembalikan 400 Bad Request untuk menunjukkan kesalahan pada isi request.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /movies
Header	x-api-key: SI5C-KUIS1-2428240113 Content-Type: application/json
Body	JSON tidak valid / struktur body tidak ditutup dengan benar.
Hasil yang diharapkan	400 Bad Request; response tetap JSON.
Status hasil uji	400 Bad Request — pesan: "JSON request tidak valid".
Lulus?	[X] Ya [] Tidak



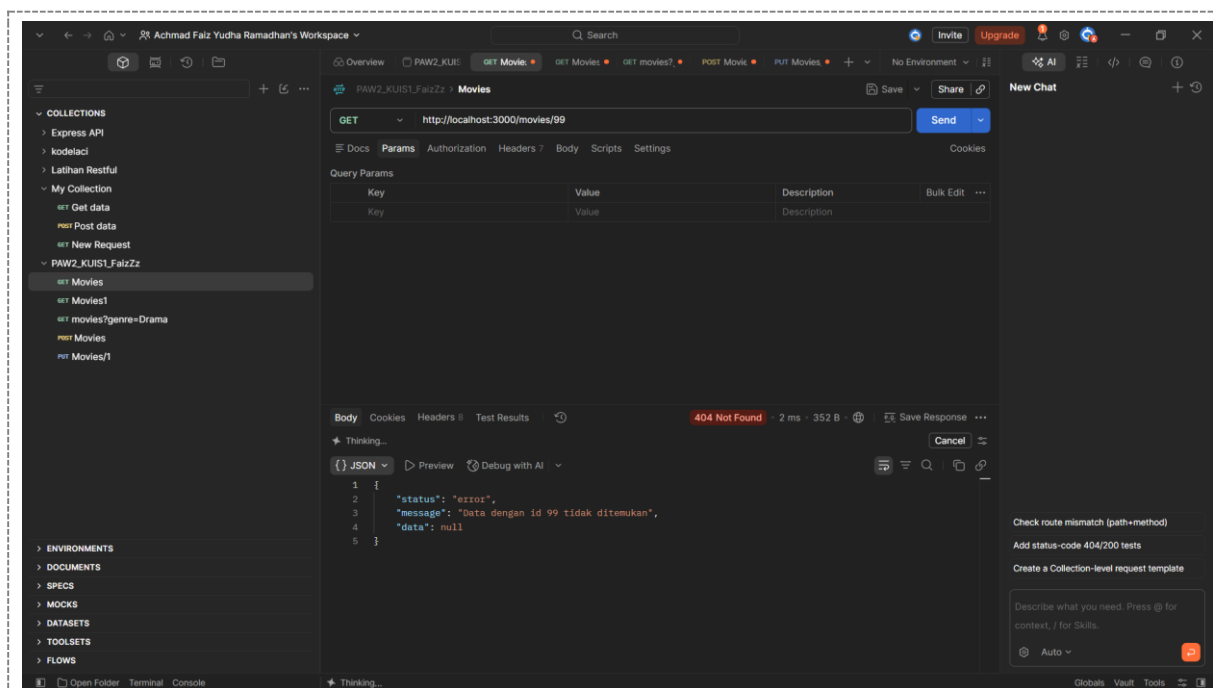
Gambar 11. POST dengan JSON rusak

Penjelasan: JSON rusak dideteksi oleh parser JSON milik Express ketika `express.json()` membaca body. Error kemudian diteruskan ke errorHandler yang mengubahnya menjadi response JSON 400 dengan pesan "JSON request tidak valid".

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /movies/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found — "Data dengan id 99 tidak ditemukan".
Lulus?	[X] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan: Controller membaca id melalui req.params dan meminta data ke movieModel.getMovieById. Karena id 99 tidak ditemukan, controller mengirim status 404 dengan pesan JSON bahwa data tidak ditemukan.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	Ya
T02	Menampilkan satu data berdasarkan ID	200	200	Ya
T03	Menambah data baru	201	201	Ya
T04	Mengubah data	200	200	Ya
T05	Menghapus data	204	200	Ya
N01	Menambah data tanpa API key	401	401	Ya
N02	Menambah data dengan body tidak lengkap	400	400	Ya
N03	Mengirim JSON yang rusak	400	400	Ya
N04	Mengambil ID yang tidak ada	404	404	Ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/FaizZzhue/si5c_kuis1_achmadfaizyudharamadhan (sesuaikan jika URL repo akhir berbeda)
Pesan commit terakhir	docs: add kuis refactor documentation
Folder routes, controllers, models, middlewares ada	☒ Ya
.env tidak terunggah (ada di .gitignore)	☒ Ya
.env.example ada	☒ Ya

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan: Route bertugas memetakan alamat dan method HTTP ke controller, misalnya `router.get("/", movieController.getMovies)`. Controller menangani request, validasi, dan response, contohnya `const getMovies = (req, res) => {...}`. Model menyimpan data dan logika pengolahannya tanpa req/res, contohnya `const getAllMovies = () => movies`.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan: Error handler terpusat membuat penanganan kesalahan konsisten tanpa harus menulis format response error yang sama pada setiap route. Buktinya terlihat pada pengujian JSON rusak, ketika error dari `express.json()` ditangkap errorHandler dan diubah menjadi response JSON 400 dengan pesan "JSON request tidak valid", bukan halaman HTML atau stack trace.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	[Ya]
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	[Ya]
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	[Ya]
4	Identitas saya (nama atau NIM) muncul pada data uji POST	[Ya]
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	[Ya]
6	Tabel rekapitulasi sesuai dengan tangkapan layar	[Ya]
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggal	[Ya]
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	[Ya]

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100